



Complete Study Notes Data Analysis & Manipulation with Python

[Series](#) · [DataFrame](#) · [Indexing](#) · [GroupBy](#) · [Merging](#) · [Time Series](#) · [Visualization](#)

[Data Science Course Preparation Guide](#)

■ Table of Contents

1. Introduction to Pandas

- What is Pandas?
- Installation & Import
- Core Data Structures

2. Series

- Creating Series
- Series Attributes & Methods
- Indexing & Slicing Series

3. DataFrame

- Creating DataFrames
- DataFrame Attributes
- Viewing Data

4. Selecting & Filtering Data

- loc & iloc
- Boolean Filtering
- query() Method

5. Data Cleaning

- Handling Missing Values
- Duplicates
- Data Type Conversion
- String Operations

6. Adding, Modifying & Deleting

- Columns & Rows
- apply() & map()
- Lambda Functions

7. Sorting & Ranking

- sort_values, sort_index
- rank()
- nlargest / nsmallest

8. GroupBy & Aggregation

- groupby()
- agg(), transform()
- pivot_table()

9. Merging, Joining & Concatenating

- merge()
- join()
- concat()

10. Reshaping Data

- melt()
- pivot()
- stack() & unstack()
- crosstab()

11. Time Series

- DatetimeIndex
- resampling
- rolling()
- date_range()

12. File I/O

- CSV, Excel, JSON, SQL, Parquet

13. DataFrame Styling & Display Options

14. Performance Tips

- memory_usage
- categorical dtype
- vectorization

15. Quick Reference Cheatsheet

1. Introduction to Pandas

What is Pandas?

Pandas is an open-source Python library for **data manipulation and analysis**. Built on top of NumPy, it provides labeled data structures — Series and DataFrame — that make working with structured/tabular data fast, expressive, and intuitive. It is the most widely used tool in data science for data wrangling, cleaning, exploration, and preparation.

- Created by Wes McKinney at AQR Capital Management (2008)
- Name comes from **panel data** (econometrics term)
- Handles CSV, Excel, SQL, JSON, Parquet, HDF5 and more
- Integrates tightly with NumPy, Matplotlib, Scikit-learn
- Key for **EDA** (Exploratory Data Analysis)

Installation & Import

```
pip install pandas
import pandas as pd
import numpy as np # Usually used together
```

Core Data Structures

Structure	Dimensions	Analogy	Use Case
Series	1D	Column in spreadsheet	Single variable / time series
DataFrame	2D	Full spreadsheet / SQL table	Tabular dataset (rows × columns)
Index	N/A	Row labels	Identifies each row

2. Series

A Series is a **one-dimensional labeled array** capable of holding any data type. It has both a data array and an associated array of labels called the **index**.

Creating Series

```
s = pd.Series([10, 20, 30, 40]) # default int index 0,1,2,3
s = pd.Series([10,20,30], index=['a','b','c']) # custom index
s = pd.Series({'a':1, 'b':2, 'c':3}) # from dict
s = pd.Series(5, index=[0,1,2,3]) # scalar broadcast
s = pd.Series(np.arange(5)) # from NumPy array
```

Series Attributes & Methods

Attribute/Method	Description
s.values	NumPy array of values
s.index	Index object
s.dtype	Data type
s.shape	Tuple (n,)
s.size	Number of elements
s.name	Name of the Series
s.head(n)	First n elements (default 5)
s.tail(n)	Last n elements
s.describe()	Summary statistics
s.unique()	Array of unique values
s.nunique()	Count of unique values
s.value_counts()	Frequency of each value
s.sort_values()	Sort by value
s.isnull()	Boolean mask for NaN

Indexing & Slicing Series

```
s = pd.Series([10,20,30,40], index=['a','b','c','d'])
s['b'] # 20 – label-based
s[1] # 20 – positional
s[['a','c']] # multiple labels
s['a':'c'] # label slice (inclusive!)
s[s > 15] # boolean filter → [20 30 40]
s.iloc[1:3] # positional slice
```

3. DataFrame

A DataFrame is a **two-dimensional, size-mutable, labeled data structure** with columns of potentially different types. Think of it as a dict of Series objects sharing the same index.

Creating DataFrames

```
# From dict of lists
df = pd.DataFrame({'name': ['Alice', 'Bob', 'Carol'], 'age': [25, 30, 35],
                  'score': [85.0, 90.5, 78.0]})

# From list of dicts
df = pd.DataFrame([{'a': 1, 'b': 2}, {'a': 3, 'b': 4}])

# From NumPy array
df = pd.DataFrame(np.random.randn(5, 3), columns=['A', 'B', 'C'])

# From CSV (most common)
df = pd.read_csv('data.csv')

# Specifying index
df = pd.DataFrame(data, index=['r1', 'r2', 'r3'])
```

DataFrame Attributes

Attribute	Description	Example Output
df.shape	(rows, cols) tuple	(100, 5)
df.dtypes	dtype of each column	col1 int64 ...
df.columns	Column labels	Index(['a', 'b'])
df.index	Row labels	RangeIndex(0,100)
df.values	NumPy array of data	array([[...]])
df.size	Total elements	500
df.ndim	Number of dimensions	2
df.memory_usage()	Memory per column (bytes)	col1 800 ...

Viewing & Inspecting Data

```
df.head(10) # first 10 rows (default 5)
df.tail(5) # last 5 rows
df.sample(5) # 5 random rows
df.info() # column types, non-null counts, memory
df.describe() # stats: count/mean/std/min/quartiles/max
df.describe(include='all') # include non-numeric cols
df.shape # (rows, cols)
df.isnull().sum() # missing values per column
df.duplicated().sum() # count duplicate rows
df.nunique() # unique values per column
df.value_counts() # frequency table (Series)
```

```
df.corr() # correlation matrix  
df.cov() # covariance matrix
```

4. Selecting & Filtering Data

Column Selection

```
df['age'] # single column → Series
df[['name','age']] # multiple columns → DataFrame
df.age # dot notation (careful with spaces/keywords!)
```

loc — Label-Based Selection

```
df.loc[2] # row with label 2
df.loc[0:3] # rows 0,1,2,3 (INCLUSIVE)
df.loc[0:3, 'name'] # rows 0-3, column 'name'
df.loc[:, 'age':'score'] # all rows, columns age to score
df.loc[df['age'] > 28] # boolean selection
df.loc[[0,2,4], ['name','score']] # specific rows & cols
```

iloc — Integer Position-Based Selection

```
df.iloc[0] # first row
df.iloc[-1] # last row
df.iloc[0:3] # rows 0,1,2 (NOT inclusive at end)
df.iloc[0, 1] # row 0, col 1
df.iloc[:, 2] # all rows, column index 2
df.iloc[[0,2,4], [0,2]] # specific rows and column positions
df.iloc[:,2] # every other row
```

■ **loc** is label-based (end inclusive), **iloc** is position-based (end exclusive). Never mix them up!

Boolean Filtering

```
df[df['age'] > 28] # age > 28
df[(df['age'] > 25) & (df['score'] > 80)] # AND condition
df[(df['age'] < 25) | (df['score'] > 90)] # OR condition
df[~(df['age'] > 28)] # NOT condition
df[df['name'].isin(['Alice','Bob'])] # isin()
df[df['name'].notna()] # exclude NaN
df[df['age'].between(25, 35)] # between inclusive
```

query() Method

```
df.query('age > 28') # string expression
df.query('age > 28 and score > 80') # compound
df.query('name in ["Alice", "Bob"]') # list check
threshold = 28
df.query('age > @threshold') # use Python variable with @
```

5. Data Cleaning

Handling Missing Values

```
df.isnull() # True where NaN
df.notnull() # True where not NaN
df.isnull().sum() # count NaN per column
df.isnull().sum().sum() # total NaN in df

# Dropping
df.dropna() # drop rows with ANY NaN
df.dropna(axis=1) # drop columns with ANY NaN
df.dropna(how='all') # drop only if ALL values are NaN
df.dropna(subset=['age', 'score']) # check only these columns
df.dropna(thresh=3) # keep rows with >= 3 non-NaN

# Filling
df.fillna(0) # fill all NaN with 0
df.fillna({'age':0, 'name':'Unknown'}) # fill per column
df['age'].fillna(df['age'].mean()) # fill with column mean
df.fillna(method='ffill') # forward fill
df.fillna(method='bfill') # backward fill
df['age'].interpolate() # linear interpolation
```

Removing Duplicates

```
df.duplicated() # boolean Series
df.duplicated(subset=['name']) # check specific columns
df.duplicated(keep='last') # mark first as duplicate
df.drop_duplicates() # remove duplicate rows
df.drop_duplicates(subset=['name']) # based on specific col
df.drop_duplicates(keep='last', inplace=True)
```

Data Type Conversion

```
df['age'].astype(int) # convert to int
df['score'].astype(float) # to float
df['col'].astype('category') # to categorical (saves memory)
pd.to_numeric(df['col'], errors='coerce') # coerce invalid to NaN
pd.to_datetime(df['date']) # parse date strings
pd.to_datetime(df['date'], format='%Y-%m-%d')
df.convert_dtypes() # infer best dtypes automatically
```

String Operations (str accessor)

```
df['name'].str.lower() # lowercase
df['name'].str.upper() # uppercase
df['name'].str.strip() # remove whitespace
df['name'].str.replace('old', 'new') # find & replace
df['name'].str.contains('ali', case=False) # search
df['name'].str.startswith('A') # starts with
df['name'].str.split(' ') # split into list
df['name'].str.split(' ', expand=True) # into columns
df['name'].str.len() # string length
df['name'].str.extract(r'(\w+)') # regex extract
```

6. Adding, Modifying & Deleting Data

Columns

```
# Add new column
df['grade'] = 'A' # constant value
df['bonus'] = df['score'] * 0.1 # computed
df.insert(2, 'rank', [1,2,3]) # insert at position 2

# Rename columns
df.rename(columns={'name':'Name', 'age':'Age'}, inplace=True)
df.columns = ['Name', 'Age', 'Score'] # replace all
df.columns = df.columns.str.lower() # lowercase all

# Delete column
df.drop('grade', axis=1, inplace=True) # drop one
df.drop(['col1', 'col2'], axis=1) # drop multiple
del df['col'] # alternative
df.pop('col') # returns & removes
```

Rows

```
# Add row
new_row = pd.DataFrame([{'name':'Dave', 'age':28, 'score':95}])
df = pd.concat([df, new_row], ignore_index=True)

# Delete rows
df.drop(0, axis=0) # drop row with label 0
df.drop([0, 1, 2], axis=0) # drop multiple rows
df.drop(df[df['age'] < 20].index) # conditional drop

# Reset index after dropping
df.reset_index(drop=True, inplace=True)
```

apply(), map(), applymap()

```
# apply() – works on Series or DataFrame
df['age'].apply(lambda x: x + 1) # on column
df.apply(lambda col: col.max() - col.min()) # on each column
df.apply(lambda row: row['age']*2, axis=1) # on each row

# map() – element-wise on Series (replaces values)
df['grade'] = df['score'].map({90:'A', 80:'B', 70:'C'})
df['name'].map(str.upper) # function mapping

# applymap() – element-wise on whole DataFrame (deprecated → use map)
df.map(lambda x: round(x,2) if isinstance(x,float) else x)
```

7. Sorting & Ranking

```
df.sort_values('age') # sort by age ascending
df.sort_values('age', ascending=False) # descending
df.sort_values(['age', 'score'], ascending=[True, False]) # multi-key
df.sort_values('name', na_position='last') # NaN at end

df.sort_index() # sort by row index
df.sort_index(axis=1) # sort by column names

df['rank'] = df['score'].rank() # default: average for ties
df['score'].rank(method='min') # min rank for ties
df['score'].rank(method='dense') # dense ranking
df['score'].rank(ascending=False) # highest score = rank 1

df.nlargest(5, 'score') # top 5 by score
df.nsmallest(3, 'age') # bottom 3 by age
```

8. GroupBy & Aggregation

groupby() Basics

```
# Basic groupby
g = df.groupby('dept') # GroupBy object
df.groupby('dept')['salary'].mean() # mean salary per dept
df.groupby('dept').size() # count rows per group
df.groupby('dept').count() # non-NaN count per column

# Multi-column groupby
df.groupby(['dept', 'level'])['salary'].mean()

# Iterate groups
for name, group in df.groupby('dept'):
    print(name, group.shape)
```

Aggregation Functions

```
# Built-in aggregation methods
df.groupby('dept')['salary'].agg(['mean', 'min', 'max', 'std', 'count'])

# Custom agg with dict
df.groupby('dept').agg({
    'salary': ['mean', 'sum'],
    'age': ['min', 'max']
})

# Named aggregation (modern style)
df.groupby('dept').agg(
    avg_sal=('salary', 'mean'),
    max_age=('age', 'max'),
    count=('name', 'count')
)

# Custom function
df.groupby('dept')['salary'].agg(lambda x: x.max() - x.min())
```

transform() — Keep Original Shape

```
# transform returns same-shape result (useful for adding group stats to original df)
df['dept_avg'] = df.groupby('dept')['salary'].transform('mean')
df['normalized'] = df.groupby('dept')['salary'].transform(
    lambda x: (x - x.mean()) / x.std()
) # z-score within group
```

pivot_table()

```
pd.pivot_table(df,
    values='salary', # values to aggregate
    index='dept', # row grouping
    columns='level', # column grouping
    aggfunc='mean', # or 'sum', 'count', np.mean
    fill_value=0, # fill missing combinations
    margins=True # add row/col totals
```

)

9. Merging, Joining & Concatenating

merge() — SQL-style Joins

```
# Inner join (default) – only matching keys
pd.merge(df1, df2, on='id')

# Left join – all rows from df1
pd.merge(df1, df2, on='id', how='left')

# Right join – all rows from df2
pd.merge(df1, df2, on='id', how='right')

# Outer join – all rows from both
pd.merge(df1, df2, on='id', how='outer')

# Different key names
pd.merge(df1, df2, left_on='emp_id', right_on='id')

# Join on index
pd.merge(df1, df2, left_index=True, right_index=True)

# Suffixes for duplicate column names
pd.merge(df1, df2, on='id', suffixes=('_left', '_right'))
```

concat() — Stack DataFrames

```
pd.concat([df1, df2]) # stack vertically (rows)
pd.concat([df1, df2], axis=1) # stack horizontally (cols)
pd.concat([df1, df2], ignore_index=True) # reset index
pd.concat([df1, df2], keys=['a', 'b']) # hierarchical index
pd.concat([df1, df2], join='inner') # only common columns
pd.concat([df1, df2], verify_integrity=True) # raise on dupe index
```

join() & combine_first()

```
df1.join(df2) # join on index
df1.join(df2, how='outer') # outer join on index
df1.join(df2, on='key') # join df1 key to df2 index

df1.combine_first(df2) # fill NaN in df1 from df2
```

Join Types Summary

Type	How	Result
Inner	how='inner'	Only rows with matching keys in BOTH frames
Left	how='left'	All rows from LEFT; NaN for non-matching RIGHT
Right	how='right'	All rows from RIGHT; NaN for non-matching LEFT
Outer	how='outer'	All rows from BOTH; NaN where no match
Cross	how='cross'	Cartesian product of all rows

10. Reshaping Data

melt() — Wide to Long Format

```
# Wide: one row per person, columns are years
# Long: one row per (person, year)
pd.melt(df,
id_vars=['name'], # columns to keep as-is
value_vars=['2022', '2023'], # columns to melt
var_name='year', # new column for old headers
value_name='sales' # new column for values
)
```

pivot() — Long to Wide Format

```
df.pivot(index='name', columns='year', values='sales')
# Each unique 'year' becomes a column
# Raises error if duplicates exist → use pivot_table() instead
```

stack() & unstack()

```
# stack(): moves columns → innermost row index
df_stacked = df.stack()

# unstack(): moves innermost row index → columns
df_unstacked = df_stacked.unstack()
df_unstacked = df_stacked.unstack(level=0) # specify level
```

crosstab() & cut()

```
# Frequency cross-tabulation
pd.crosstab(df['dept'], df['level'])
pd.crosstab(df['dept'], df['level'], values=df['salary'], aggfunc='mean')

# Binning continuous data
pd.cut(df['age'], bins=3) # equal-width bins
pd.cut(df['age'], bins=[0,18,35,60,100],
labels=['youth', 'young', 'middle', 'senior']) # custom bins
pd.qcut(df['age'], q=4) # equal-frequency (quartile) bins
```

11. Time Series

Creating Date Ranges & Parsing Dates

```
pd.date_range('2024-01-01', periods=12, freq='M') # monthly
pd.date_range('2024-01-01', '2024-12-31', freq='D') # daily
pd.date_range('2024-01-01', periods=8, freq='H') # hourly

# Frequency aliases: D=day, W=week, M=month end, MS=month start,
# Q=quarter end, A=year end, H=hour, T=minute, S=second

pd.to_datetime('2024-03-15') # parse string
pd.to_datetime(df['date_col']) # parse column
pd.to_datetime(df['date'], format='%d/%m/%Y') # custom format
```

DatetimeIndex Operations

```
df = df.set_index('date') # set date as index

# Accessing date components
df.index.year / .month / .day / .hour
df.index.day_name() # 'Monday', 'Tuesday'...
df.index.quarter # 1-4
df.index.dayofweek # 0=Mon, 6=Sun

# Slicing with date strings
df['2024'] # all of 2024
df['2024-03'] # March 2024
df['2024-01-01':'2024-06-30'] # date range

# dt accessor (for column, not index)
df['date'].dt.year
df['date'].dt.month_name()
df['date'].dt.strftime('%B %Y') # format as string
```

Resampling & Rolling

```
# Resample: change frequency
df.resample('M').mean() # monthly average
df.resample('Q').sum() # quarterly sum
df.resample('W').agg({'price':'mean', 'volume':'sum'})

# Rolling: sliding window
df['price'].rolling(window=7).mean() # 7-day moving average
df['price'].rolling(7).std() # rolling std dev
df['price'].rolling(7, min_periods=1).mean() # allow partial windows

# Expanding: cumulative from start
df['price'].expanding().mean() # cumulative mean

# Shifting
df['price'].shift(1) # lag by 1 period
df['price'].shift(-1) # lead by 1 period
df['price'].diff(1) # first difference
```

12. File I/O

CSV

```
df = pd.read_csv('file.csv') # basic read
df = pd.read_csv('file.csv', index_col=0) # first col as index
df = pd.read_csv('file.csv', parse_dates=['date']) # parse date col
df = pd.read_csv('file.csv', usecols=['a', 'b']) # select columns
df = pd.read_csv('file.csv', nrows=100) # only first 100 rows
df = pd.read_csv('file.csv', skiprows=2) # skip first 2 rows
df = pd.read_csv('file.csv', na_values=['NA', '--']) # custom NaN values
df = pd.read_csv('file.csv', dtype={'age':int}) # force dtype
df = pd.read_csv('file.csv', encoding='utf-8') # specify encoding
df = pd.read_csv('file.csv', sep=';') # semicolon delimiter

df.to_csv('out.csv', index=False) # save without index
df.to_csv('out.csv', float_format='%.2f') # format floats
```

Excel

```
df = pd.read_excel('file.xlsx') # first sheet
df = pd.read_excel('file.xlsx', sheet_name='Sheet2')
dfs = pd.read_excel('file.xlsx', sheet_name=None) # all sheets → dict

df.to_excel('out.xlsx', index=False)
with pd.ExcelWriter('out.xlsx') as w:
    df1.to_excel(w, sheet_name='Data')
    df2.to_excel(w, sheet_name='Summary')
```

JSON, SQL & Parquet

```
# JSON
df = pd.read_json('file.json')
df.to_json('out.json', orient='records')

# SQL
import sqlalchemy
engine = sqlalchemy.create_engine('sqlite:///db.sqlite')
df = pd.read_sql('SELECT * FROM table', engine)
df = pd.read_sql_table('tablename', engine)
df.to_sql('tablename', engine, if_exists='replace', index=False)

# Parquet (fast columnar format)
df = pd.read_parquet('file.parquet')
df.to_parquet('out.parquet', index=False)

# Pickle (Python binary – preserves dtypes)
df.to_pickle('file.pkl'); df = pd.read_pickle('file.pkl')
```

13. Display Options & Styling

```
pd.set_option('display.max_rows', 100) # show more rows
pd.set_option('display.max_columns', 50) # show more columns
pd.set_option('display.float_format', '{:.2f}'.format) # format floats
pd.set_option('display.max_colwidth', 50) # column width
pd.reset_option('all') # reset all options

# Styling (Jupyter)
df.style.highlight_max(color='lightgreen') # highlight max values
df.style.highlight_null(null_color='red') # highlight NaN
df.style.background_gradient(cmap='Blues') # heatmap style
df.style.format({'salary': '${:,.2f}'}) # format columns
df.style.bar(subset=['score']) # bar chart in cells
```

Performance Tips

```
# Use categorical dtype for low-cardinality string columns
df['dept'] = df['dept'].astype('category') # huge memory saving

# Vectorized operations over loops
df['total'] = df['price'] * df['qty'] # fast

# Avoid iterrows() – very slow!
# SLOW: for i, row in df.iterrows(): ...
# FAST: df.apply(func, axis=1) or vectorized ops

# Check memory usage
df.memory_usage(deep=True).sum() / 1024**2 # in MB

# Read large files in chunks
for chunk in pd.read_csv('big.csv', chunksize=10000):
    process(chunk)

# Use eval() for complex expressions
df.eval('profit = revenue - cost', inplace=True)
```

15. Quick Reference Cheatsheet

Creating

Code	Description
<code>pd.Series([1,2,3])</code>	Create Series
<code>pd.DataFrame(dict)</code>	Create DataFrame
<code>pd.read_csv('f.csv')</code>	Load CSV
<code>pd.read_excel('f.xlsx')</code>	Load Excel

Inspection

Code	Description
<code>df.head()</code> / <code>.tail()</code>	First/last rows
<code>df.info()</code>	Column types & nulls
<code>df.describe()</code>	Summary statistics
<code>df.shape</code> / <code>.dtypes</code>	Dimensions / types
<code>df.isnull().sum()</code>	Count missing values

Selection

Code	Description
<code>df['col']</code>	Single column
<code>df[['c1','c2']]</code>	Multiple columns
<code>df.loc[rows, cols]</code>	Label-based
<code>df.iloc[r, c]</code>	Position-based
<code>df[df['col'] > x]</code>	Boolean filter
<code>df.query('col > x')</code>	Query string

Cleaning

Code	Description
<code>df.dropna()</code>	Drop NaN rows
<code>df.fillna(val)</code>	Fill NaN
<code>df.drop_duplicates()</code>	Remove duplicates
<code>df['col'].astype(dtype)</code>	Type conversion
<code>df['col'].str.strip()</code>	String clean

Transformation

Code	Description
<code>df['col'].apply(fn)</code>	Apply function

<code>df.sort_values('col')</code>	Sort by column
<code>df.rename(columns={})</code>	Rename columns
<code>df.drop('col',axis=1)</code>	Drop column
<code>df.reset_index()</code>	Reset index

Aggregation

Code	Description
<code>df.groupby('col').mean()</code>	Group & mean
<code>df.groupby('c').agg({...})</code>	Custom agg
<code>pd.pivot_table(df,...)</code>	Pivot table
<code>df.resample('M').sum()</code>	Resample by month
<code>df.rolling(7).mean()</code>	Rolling average

Merging

Code	Description
<code>pd.merge(df1,df2,on='id')</code>	Inner join
<code>pd.merge(...,how='left')</code>	Left join
<code>pd.concat([df1,df2])</code>	Stack vertically
<code>pd.concat(...,axis=1)</code>	Stack horizontally

Export

Code	Description
<code>df.to_csv('f.csv',index=False)</code>	Save CSV
<code>df.to_excel('f.xlsx')</code>	Save Excel
<code>df.to_json('f.json')</code>	Save JSON
<code>df.to_parquet('f.parquet')</code>	Save Parquet

■ Next: Matplotlib → Seaborn → Scikit-learn → Machine Learning Pipelines

Official docs: pandas.pydata.org/docs | Exercises: github.com/guipsamora/pandas_exercises